

TECHNICAL ASSESSMENT

AsiaPacTalents · Python · FastAPI · SQLite · LLM Integration | Format: Mini Project | Time: ~1 hours

OVERVIEW

Build a small backend system that simulates a WhatsApp-style chat. You are not building a real app — just a backend API that receives a message, decides who should handle it, generates a reply, and saves everything to a database.

You can use AI coding tools. The goal is a working system, not perfect code.

WHAT YOU ARE BUILDING

A simple API (backend only — no frontend needed) that:

- Receives an incoming chat message
- Decides which agent type should reply (sales, support, or general)
- Generates a reply using an AI model or a simple mock function
- Saves the message and reply to a database
- Lets you view saved conversations

REQUIREMENTS

1. Incoming Message Endpoint

Create an endpoint that accepts incoming messages:

```
POST /webhook/whatsapp
```

It should accept JSON like this:

```
{
  "event_id": "evt_001",
  "user_id": "user_123",
  "message": "I want to know the price"
}
```

Protect this endpoint with a bearer token. In your .env file, set a secret token. Any request must include it in the header like this:

```
Authorization: Bearer your_secret_token
```

Requests without a valid token should be rejected with a 401 error. Document the token in your README.

2. Simulation Endpoint

Create a helper endpoint so you can easily demo the flow:

```
POST /simulate
```

This sends a test message into your own system without needing a real WhatsApp integration. Useful for the interview demo.

3. Agent Routing

When a message comes in, assign it to one of three agent types based on what the message says:

- **sales** — message mentions buying, pricing, or product interest
- **support** — message mentions a problem, issue, or complaint
- **general** — anything else

Simple keyword matching is fine. No need for anything complex.

4. Reply Generation

Use any LLM of your choice, or a simple mock function if you prefer. If the LLM fails, fall back to a default reply. Examples:

- **sales** → "Thanks for your interest. Can you share what you are looking for?"
- **support** → "Sorry you are facing this issue. Can you describe it a bit more?"
- **general** → "Happy to help. What would you like to know?"

5. Database

Use SQLite or PostgreSQL — whichever you are comfortable with. Store at least these two tables:

sessions

- id
- user_id
- agent (which agent handled it)
- last_active

messages

- id
- session_id
- role (user or assistant)
- content (the message text)
- event_id

6. Duplicate Handling

If the same event_id is sent twice, do not save it twice. A simple unique constraint on the event_id column is enough.

7. View Saved Data

Create two read endpoints:

```
GET /sessions
```

Returns a list of all sessions.

```
GET /sessions/{session_id}/messages
```

Returns all messages for a given session.

WHAT WE WILL TEST

In the interview, we will ask you to:

1. Start the project
2. Trigger a message using /simulate
3. Show that a reply was generated
4. Show that the conversation was saved to the database
5. Show /sessions and /sessions/{id}/messages working
6. Walk us through the flow in plain language
7. Make one small live change to the code

SUBMISSION

Please provide:

- A project folder or GitHub repo link
- A short README with run instructions
- A .env.example file showing which environment variables are needed (including the bearer token)

WHAT WE CARE ABOUT

We care about:

- Does it run without errors?
- Does the full flow work end to end?
- Is data actually saved to the database?
- Can you explain what you built?

We do NOT care about:

- Perfect code style
- Complex design patterns
- Whether every line was written manually

EXPECTED TIME

Within 1 hour using AI coding tools.

NOTES

- No frontend required
- No real WhatsApp integration required
- Keep it simple and working